

DOGFOOD

Build the Platform That Will Judge You · 2026

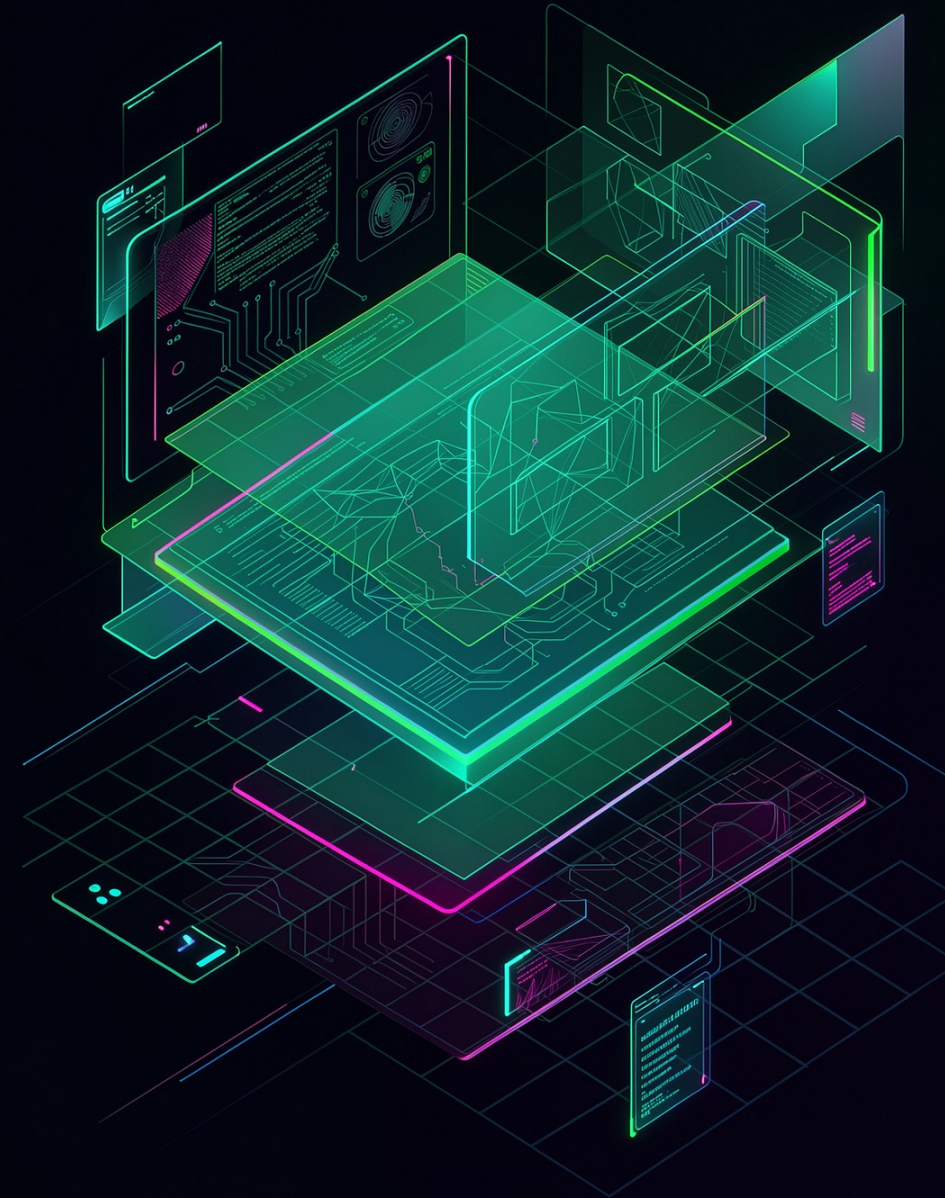
Every hackathon platform in the world converged on the same nine features, and then stopped moving. This is the hackathon where you build the replacement.

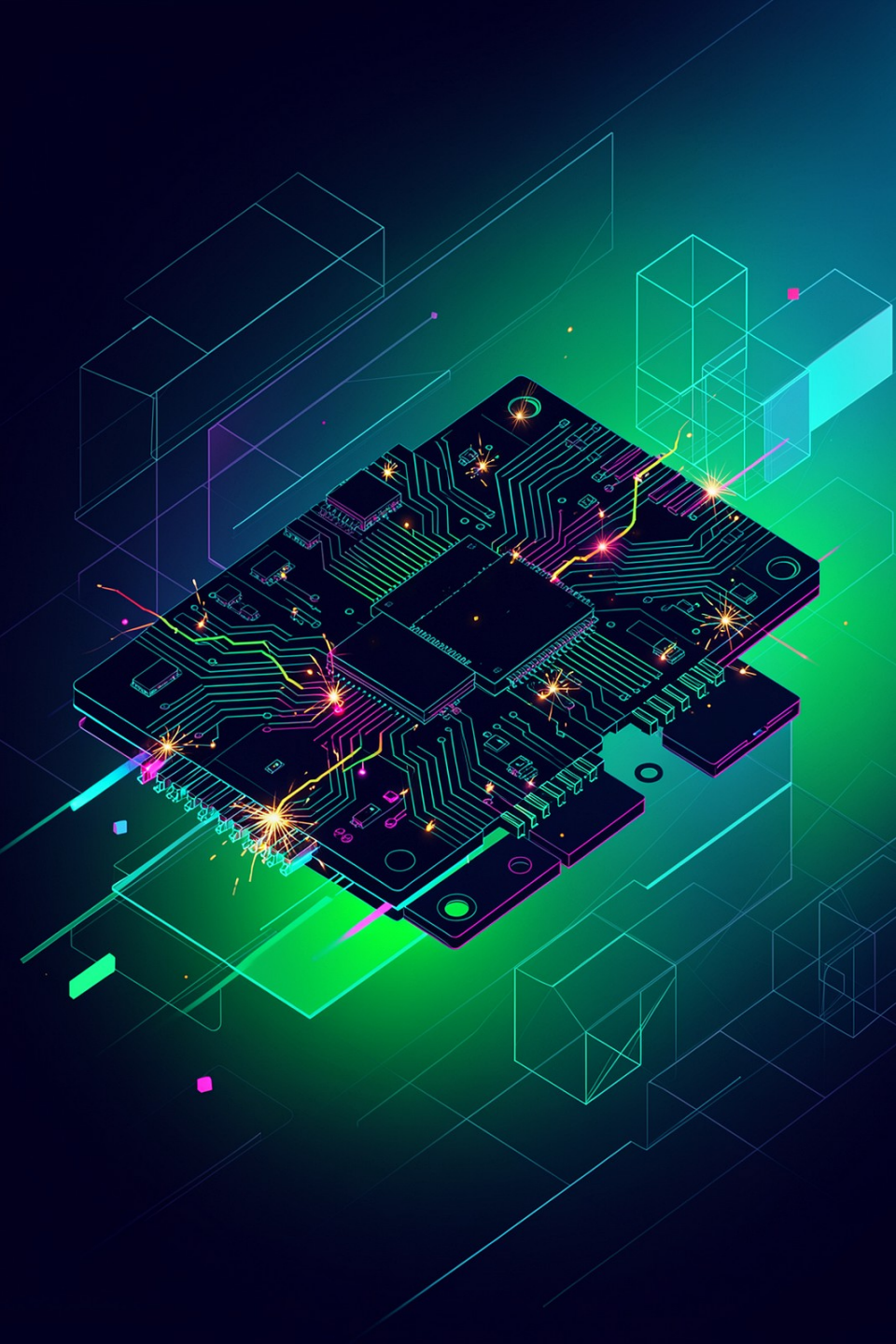
 **Sep 26–29, 2026**

 **Online**

 **Free**

 **\$2,500 Prize Pool**





THE PROBLEM

Broken Judging

The category leader cannot weight judging criteria. Its own docs tell organizers to judge offline in a spreadsheet.

Opaque Normalization

Multiple platforms advertise "automatic score normalization" and none of them will tell you how it works.

Gameable Voting

Community voting is universally conceded to be gameable. The standard advice is to keep the prize small and hide results.

No API. Ever.

After fifteen years and millions of developers, not one of them ships a public API. The integration layer of an entire industry is scrapers and downloaded CSVs.



You are building the thing that replaces all of it.

YOUR MISSION

Build a hackathon submission and judging portal. One product. Not a mockup, not a design system, not a slide deck. Working software an organizer could run on Monday.

Registration, teams, submissions, eligibility, judge assignment, scoring, normalization, results, export. Ten stages, each feeding the next. Get one wrong and the part that suffers is the judging, which is the part participants came for.

docker compose up

...and it works. On a laptop. With the network off. Or we cannot adopt it, and adoption is the entire point.



THE TIER LADDER • HOW FAR CAN YOU CLIMB

No tracks. One product, four tiers. Your score is how far up you got and how cleanly you did it.

1

T1 • CORE (*floor, must clear or you are not judged*)

Auth, roles, event creation, team formation, project submission, deadline enforcement, public gallery.

2

T2 • JUDGING (*where the real engineering starts*)

Judge invitation and assignment, weighted scoring rubric, role isolation enforced in the backend, progress dashboard, cross-judge normalization, CSV export.

3

T3 • PUBLIC

Community voting, comments, results hidden until window closes, randomised ballot order, anti-abuse with an audit trail.

4

T4 • STRETCH

REST API with webhooks, certificates, verifiable judge records, embeddable gallery, bulk import and export.

📌 A clean T2 beats a broken T4. Correctness over breadth, every time.

THE ACCEPTANCE CHECKER

One Python file. Standard library only. You point it at your portal and it tells you what works.

```
python3 run.py .dogfood.toml
```

Seven checks. Three for T1, four for T2. Same program for every team, so nobody is judged more strictly than anybody else.

T3 and T4 are judged by hand. Only T1 and T2 are machine-verified.

📄 Run it early. Run it often. Commit the output whatever it says. Two honest FAIL lines read better than a README claiming everything works.

T1 Checks

- Gallery is public
- Fixture projects shown
- Closed event refuses submissions

T2 Checks

- Judge sees own scores
- Judge cannot see peer scores
- Participant blocked
- CSV export works

.dogfood.toml • HOW THE CHECKER FINDS YOUR STUFF

Six to fifteen lines at the root of your repo. You build whatever you want, call your routes whatever you want, and then write down where they ended up.

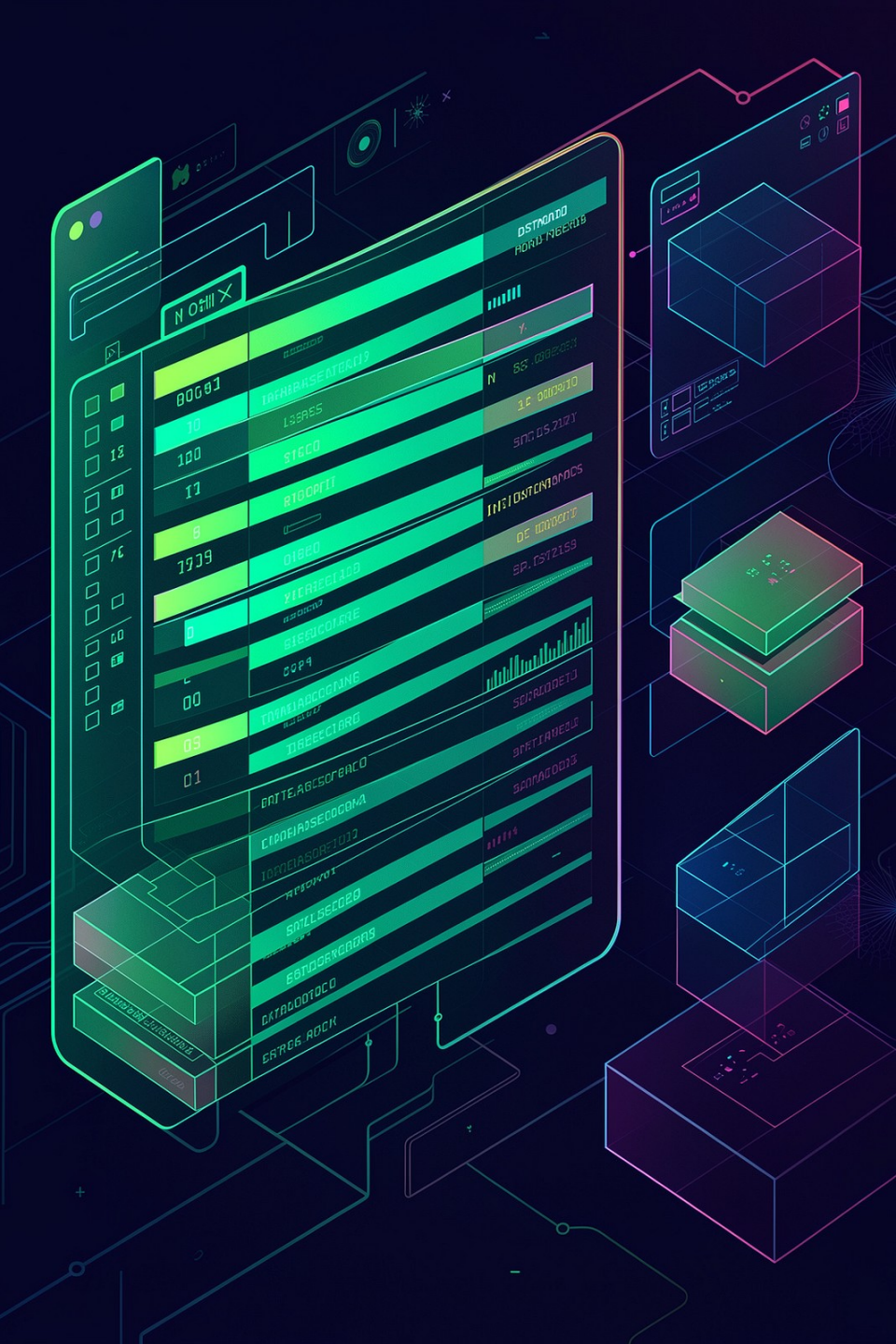
```
[portal]
base_url = "http://localhost:8080"

[tiers]
claimed = ["T1", "T2"]

[auth]
organizer = "Cookie: session=org_7f2a"
judge_a   = "Cookie: session=jdg_a_91bc"
judge_b   = "Cookie: session=jdg_b_44de"
participant = "Cookie: session=prt_2e88"

[routes]
gallery    = "/projects"
submit     = "/projects/new"
judge_scores = "/api/judge/scores"
peer_scores = "/api/judge/scores?judge=judge_a"
csv_export = "/api/export.csv"
```

The checker never logs in. You hand it a working header and it attaches it. Your login page can work however you like.



fixtures.json • THE SHARED DATASET

Every team loads the same data. 40 projects, 30 judges, 8 tracks, and a full set of scores. No real names.

It contains the awkward cases on purpose. A judge who gave everything the same score. Two review batches that were never finished. A duplicate submission. These are real things that happen at real hackathons. How your portal copes with them is interesting.

Seed from the fixture event's own `submissions_close` date. One of the checks depends on it.



Already downloadable at dogfoodhack.com/spec. Nothing is held back.

THE ONE CHECK THAT COSTS THE MOST TEAMS POINTS

A judge must not be able to read another judge's scores. Not in the UI. Over the API.

The checker sends `judge_b` to `judge_a`'s score endpoint. If it gets `200`, you fail. It needs `401` or `403`.

✗ Wrong Approach

Hiding the button is not refusing the request.

✓ Right Approach

The check has to live in the backend, because the backend is where curl arrives. One if statement.

⊗ Fix it before Sunday night, not on Sunday night.



WHAT YOU ARE SHIPPING

1 Public GitHub repo, OSI licence

MIT or Apache-2.0 preferred

2 docker compose up

To a seeded, working portal, network off

3 .dogfood.toml at the repo root

Honest tier claims

4 acceptance-report.txt committed

Whatever it says

5 README.md

What it does, how to run it, honest limits

6 ARCHITECTURE.md

The shape of the system and why

7 DATA-MODEL.md

Schema, import and export paths

8 JUDGING.md

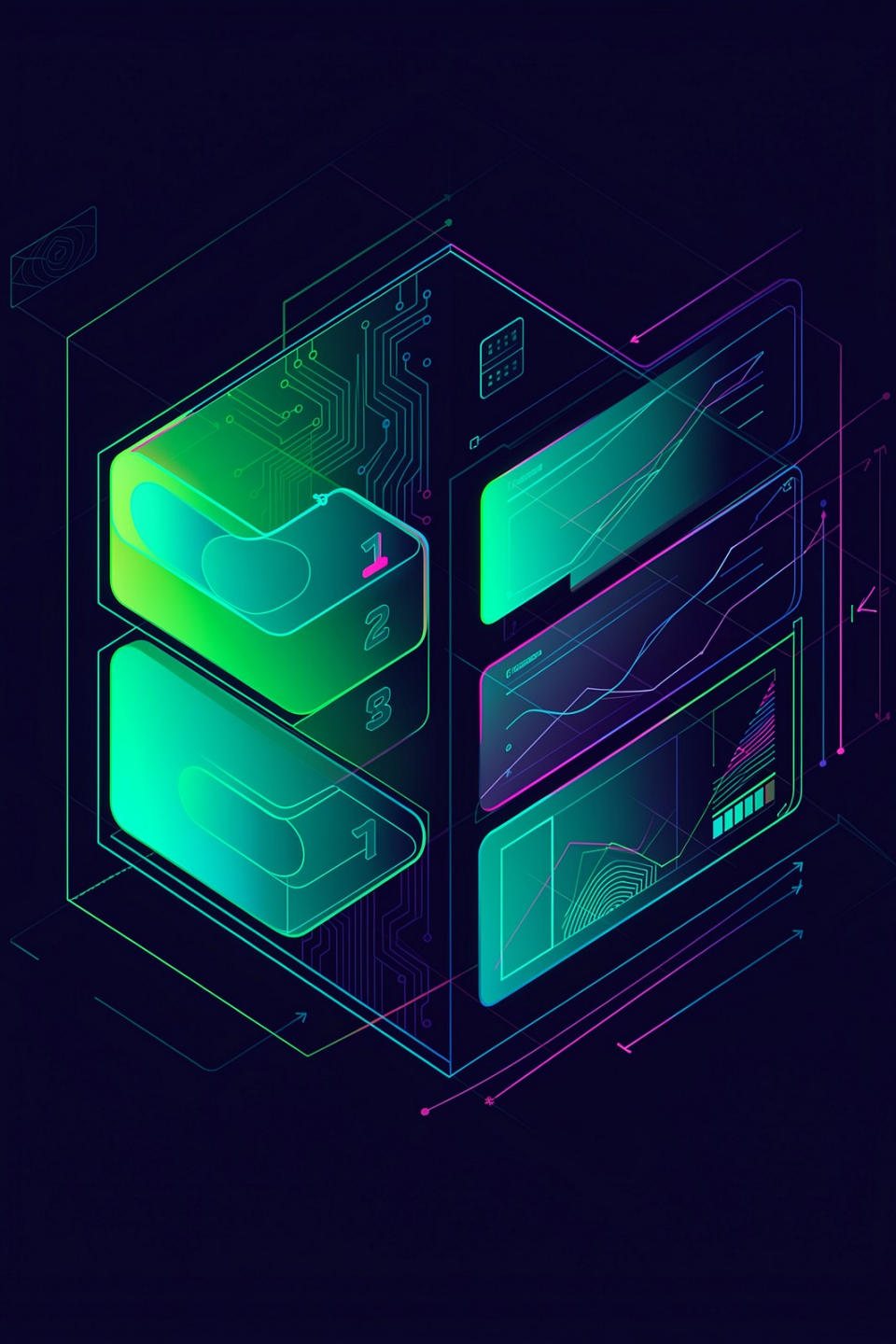
Assignment strategy, scoring maths, normalization method, defended

9 5-minute demo video

One full event lifecycle



JUDGING.md feeds straight into that 25%. "We averaged the scores" is an answer, and it is a weak one.



SCORING

40%

Tier Completion & Correctness

25%

Judging Integrity

20%

Adoptability & Operability

15%

Code Quality & Innovation

T1 is a gate, not a score. Above that, correctness beats breadth. A clean T2 outranks a T4 with three features that half-work.

📄 Honest gap reporting is rewarded. Inflated claims are penalised.

BONUS CHALLENGES • BREAK TIES, NOT YOUR SCORE

Bonus points do not change your weighted average. They break ties and decide Best Judging Engine.



Normalization Proof (+5, Hard)

Implement cross-judge normalization. Show raw, normalized, ranking change. Document it well enough that a statistician would not wince.



Pairwise Mode (+5, Hard)

Show a judge two projects, ask which is better, recover a ranking with Bradley-Terry. Sidesteps calibration entirely.



Threat Model (+3, Medium)

Sybil votes, ballot stuffing, judge collusion, deadline gaming. Name the attacks you stopped and the ones you did not.



API First (+3, Medium)

Every UI action available through a documented API with an OpenAPI spec. Nobody else in this category has one.



Pick one and nail it. Do not half-do all four.

PRIZES • \$2,500

 **Grand Prize**

\$1,200

The one we fork and run

 **Runner-Up**

\$600

 **Third Place**

\$300

 **Best Judging Engine**

\$100

 **The Teardown**

\$300

Top 3 write-ups, \$100 each

THE TEARDOWN • \$300 • TOP 3 • \$100 EACH

Publish a write-up of your build. The schema you would redo. The normalization maths that fought you. The role-isolation bug you found at hour 60. The feature you cut and do not regret.

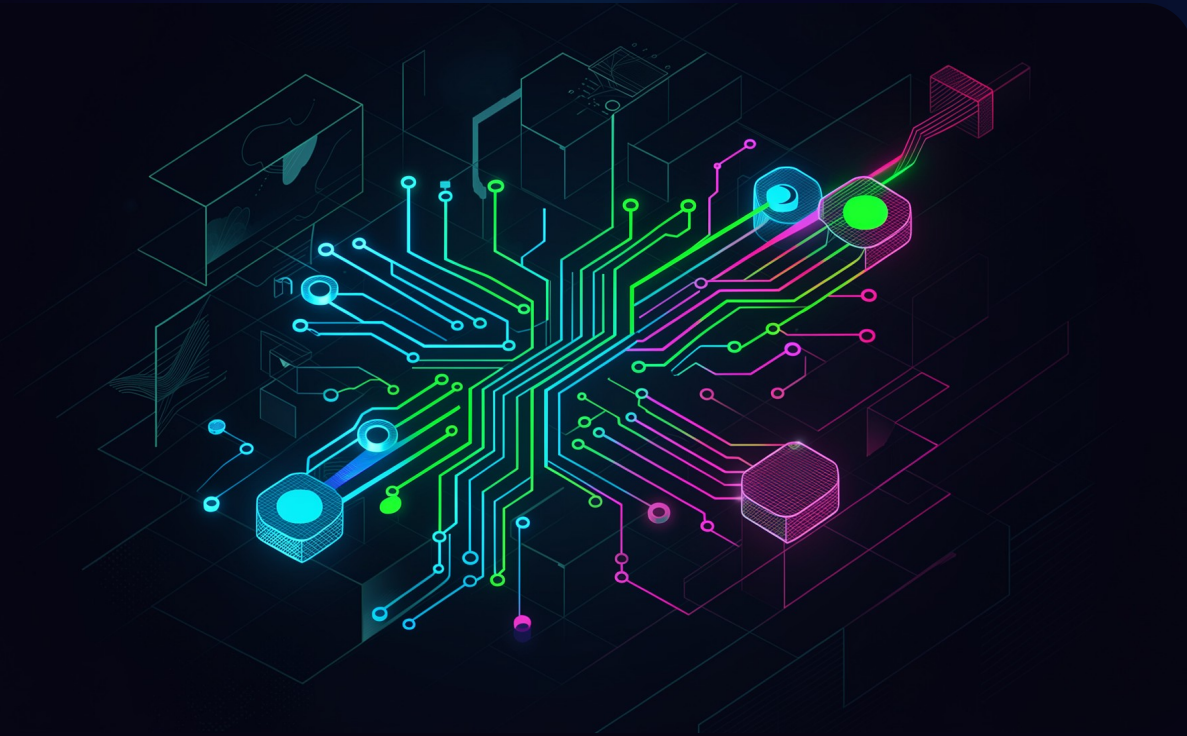
X, LinkedIn, Dev.to, your own blog. Tag Hackathon Raptors.

Judged on insight, not follower count. A 200-follower account writing something genuinely useful beats a viral thread that says nothing.

 Closes Oct 5, 2026.



WHAT HAPPENS TO THE WINNER



You keep your work. No assignment, no transfer, no CLA. Nothing to sign.

We fork it, self-host it, and run our events on it. Credit line on every event page it powers, for as long as it powers them.

Every fix and feature we add gets sent back to your repo as a pull request.

❏ This is not a maybe. We run roughly a dozen events a year. We are commissioning software, in the open, and paying for it.

TIMELINE • UTC



OUT OF SCOPE

- Design mockups or a frontend with hardcoded data behind it
- Anything that needs a cloud account, hosted database, or auth provider
- An authentication demo that stops at the login screen
- A gallery with no judging, or judging with no gallery. This is one product.
- Role checks that live only in the frontend
- LLM dumps with no architecture document and nobody able to defend the schema
- Closed source or non-OSI licence
- A rewrite of an existing platform with the name changed

BEFORE YOU START

The spec, the fixture data, and the acceptance checker are all at **dogfoodhack.com/spec**. Already downloadable. We are not holding a single file back.

Read `run.py`. It is one short file. When you finish reading it you will know every check that produces your acceptance report.

Make your seed script print the four auth headers at startup. Filling in `.dogfood.toml` then takes a minute.

Pick what you know. A boring stack you are fluent in will get further in 72 hours than an exciting one you are learning.

 **Register**

tally.so/r/2EY7zD

 **Discord**

discord.com/invite/xfYPDZYqeh

 **Spec & Fixtures**

dogfoodhack.com/spec

72 hours. One product. We run the winner.

Build the platform that will judge you.